Roteiro da apresentação

Sistemas Operacionais - Atividade 3 - UFS 2026.2

Duração-alvo: 11 minutos, com perguntas separadas de 3-5 minutos.

Texto em FALA orienta a locução; EM TELA orienta o operador. Os tempos são estimativas para ensaio. Cada slide tem um apresentador, identificado no cabeçalho, e cada integrante conclui seu bloco antes de passar a palavra.

Sequência dos 14 slides

Slide e assunto	Quem fala	Janela
01. Virtualização, namespaces e cgroups	Bruno	00:00-00:25
02. Objetivo e percurso do laboratório	Bruno	00:25-01:25
03. Ambiente e kernel compartilhado	Iasmin	01:25-02:20
04. Protocolo e controle das execuções	Iasmin	02:20-03:00
05. Namespaces: visões de processos, mounts e rede	Pedro	03:00-03:40
06. Evidências de namespaces e PIDs	Pedro	03:40-04:30
07. Carga determinística e checksum	Christian	04:30-05:15
08. Auditoria e demonstração dos registros	Christian	05:15-06:15
09. Limites efetivos do cgroup v2	Marcos	06:15-07:00
10. Throttling durante a carga	Marcos	07:00-07:45
11. Seis medições de tempo decorrido	Sibele	07:45-08:45
12. Memória: processo e cgroup	Sibele	08:45-09:25
13. Mount read-only e menor privilégio	Silas	09:25-10:15
14. Limitações, aplicação e conclusão	Silas	10:15-11:00

Participação por integrante

Integrante	Slides	Tempo-alvo
Bruno Amancio Ferreira	1, 2	01:25
Iasmin Victoria Teixeira Barreto	3, 4	01:35
Pedro César Figueiredo Carneiro	5, 6	01:30
Christian Will Silva Santos Nunes	7, 8	01:45
Marcos Vinícius Dantas Aguiar	9, 10	01:30
Sibele Oliveira Cruz	11, 12	01:40
Silas Santos da Silva	13, 14	01:35

Base: apresentacao_atividade3.pdf, 14 slides em LaTeX Beamer. Ordem: Bruno → Iasmin → Pedro → Christian → Marcos → Sibele → Silas. As instruções da demonstração e de preparação aparecem no final deste PDF.

Slide 01 - Virtualização, namespaces e cgroups

Janela 00:00-00:25 | Duração-alvo 00:25



EM TELA - não ler em voz alta

Exibir a capa com a equipe e o nome de Bruno. No vídeo, identificar Bruno na tela.

Próxima ação

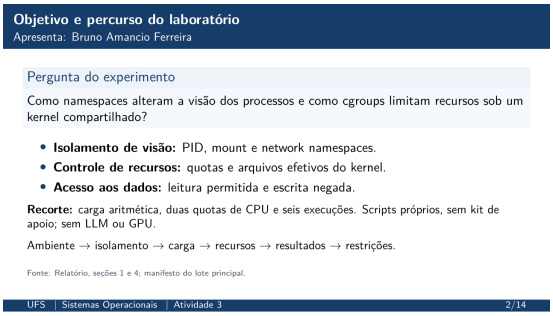
Avançar ao slide 2; Bruno continua.

FALA - Bruno Amancio Ferreira (00:00-00:25)

Olá, eu sou Bruno, da equipe GenResearch. Vamos apresentar a Atividade 3 de Sistemas Operacionais: um laboratório próprio para observar namespaces, limites de cgroups e montagem somente leitura em contêineres Linux. O trabalho reúne seis medições reais e registros que permitem conferir os resultados.

Slide 02 - Objetivo e percurso do laboratório

Janela 00:25-01:25 | Duração-alvo 01:00



EM TELA - não ler em voz alta

Apontar a pergunta e as três evidências: namespaces, limites efetivos e escrita negada. Manter Bruno até concluir o slide.

Próxima ação

Avançar ao slide 3 e passar a palavra a Iasmin Victoria Teixeira Barreto.

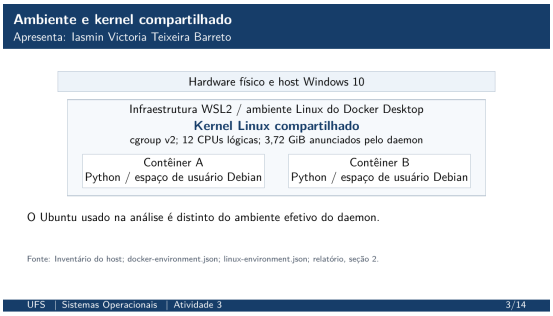
FALA - Bruno Amancio Ferreira (00:25-01:25)

A pergunta é como namespaces alteram a visão dos processos e como cgroups limitam recursos sob um kernel compartilhado. Observamos três aspectos: os identificadores de PID, mount e network; os limites efetivos de CPU, memória, swap e tarefas; e o comportamento de uma montagem somente leitura.

A carga é um cálculo aritmético em Python, sem LLM ou GPU. Comparamos meia CPU e uma CPU, usando o mesmo trabalho e três repetições por condição. Os resultados incluem os arquivos brutos, a análise e um auditor de integridade. Primeiro veremos o ambiente e o protocolo, depois o isolamento, a carga, os limites e os resultados. Silas encerra com as restrições de acesso e as limitações. Iasmin assume agora para mostrar onde os contêineres executam.

Slide 03 - Ambiente e kernel compartilhado

Janela 01:25-02:20 | Duração-alvo 00:55



EM TELA - não ler em voz alta

Iasmin assume. Percorrer o diagrama: Windows, infraestrutura WSL2, ambiente do daemon e contêineres.

Próxima ação

Avançar ao slide 4; Iasmin continua.

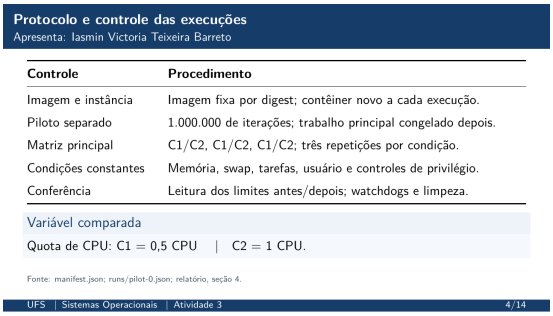
FALA - Iasmin Victoria Teixeira Barreto (01:25-02:20)

Eu sou Iasmin. O host físico utiliza Windows 10. A infraestrutura WSL2 fornece o ambiente Linux em que executa o Docker Desktop. O kernel compartilhado pertence a esse ambiente Linux intermediário. As imagens dos contêineres fornecem Python e o espaço de usuário Debian, sem iniciar um kernel próprio.

O daemon anunciou doze CPUs lógicas e aproximadamente 3,72 gibibytes de RAM. Esses recursos descrevem a camada do daemon. O Ubuntu usado na análise também deve ser distinguido desse ambiente. O diagrama mostra a camada de virtualização existente no Windows e, dentro do ambiente Linux, os dois contêineres compartilhando o kernel.

Slide 04 - Protocolo e controle das execuções

Janela 02:20-03:00 | Duração-alvo 00:40



EM TELA - não ler em voz alta

Apontar a sequência C1/C2, a imagem fixa e a leitura dos limites antes/depois. Ao terminar, passar para Pedro.

Próxima ação

Avançar ao slide 5 e passar a palavra a Pedro César Figueiredo Carneiro.

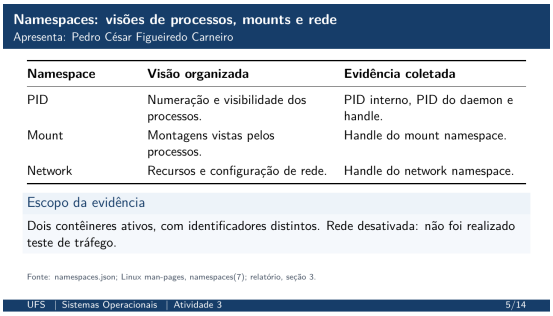
FALA - Iasmin Victoria Teixeira Barreto (02:20-03:00)

O protocolo fixa a imagem por digest e usa contêineres novos. Um piloto separado ajuda a definir a duração; depois, o trabalho fica congelado em vinte milhões de iterações. A sequência principal é C1, C2, repetida três vezes.

Mantivemos constantes memória, swap, limite de tarefas, usuário e restrições de privilégio. A quota de CPU é a condição comparada. Os limites são lidos antes e depois do trabalho, e os contêineres criados pelo laboratório são removidos ao final. Pedro vai apresentar as evidências de isolamento da visão dos processos.

Slide 05 - Namespaces: visões de processos, mounts e rede

Janela 03:00-03:40 | Duração-alvo 00:40



Namespaces: visões de processos, mounts e rede
Apresenta: Pedro César Figueiredo Carneiro

Namespace	Visão organizada	Evidência coletada
PID	Numeração e visibilidade dos processos.	PID interno, PID do daemon e handle.
Mount	Montagens vistas pelos processos.	Handle do mount namespace.
Network	Recursos e configuração de rede.	Handle do network namespace.

Escopo da evidência

Dois contêineres ativos, com identificadores distintos. Rede desativada: não foi realizado teste de tráfego.

Fonte: namespaces.json; Linux man-pages, namespaces(7); relatório, seção 3.

UFS | Sistemas Operacionais | Atividade 3 5/14

EM TELA - não ler em voz alta

Pedro assume. Relacionar cada namespace à visão que ele organiza; separar identificação de namespace de teste de tráfego.

Próxima ação

Avançar ao slide 6; Pedro continua.

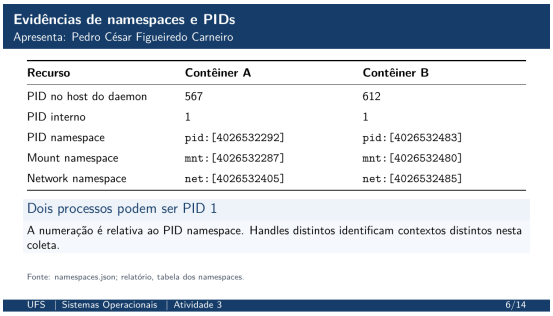
FALA - Pedro César Figueiredo Carneiro (03:00-03:40)

Eu sou Pedro. Os namespaces organizam visões de recursos do kernel. O PID namespace define a numeração de processos vista naquele contexto. O mount namespace organiza a visão das montagens, e o network namespace separa a visão de recursos de rede.

Coletamos os handles de dois contêineres ativos. Handles diferentes identificam namespaces diferentes nesta execução. Isso não demonstra isolamento absoluto nem mede tráfego de rede: a rede dos contêineres estava desativada. No próximo slide, veremos os identificadores e a relação entre o PID interno e o informado pelo daemon.

Slide 06 - Evidências de namespaces e PIDs

Janela 03:40-04:30 | Duração-alvo 00:50



Evidências de namespaces e PIDs
Apresenta: Pedro César Figueiredo Carneiro

Recurso	Contêiner A	Contêiner B
PID no host do daemon	567	612
PID interno	1	1
PID namespace	pid:[4026532292]	pid:[4026532483]
Mount namespace	mnt:[4026532287]	mnt:[4026532480]
Network namespace	net:[4026532405]	net:[4026532485]

Dois processos podem ser PID 1

A numeração é relativa ao PID namespace. Handles distintos identificam contextos distintos nesta coleta.

Fonte: namespaces.json; relatório, tabela dos namespaces.

UFS | Sistemas Operacionais | Atividade 3 6/14

EM TELA - não ler em voz alta

Apontar PIDs 567/612 no host do daemon, PID interno 1/1 e handles distintos. Ao terminar, passar para Christian.

Próxima ação

Avançar ao slide 7 e passar a palavra a Christian Will Silva Santos Nunes.

FALA - Pedro César Figueiredo Carneiro (03:40-04:30)

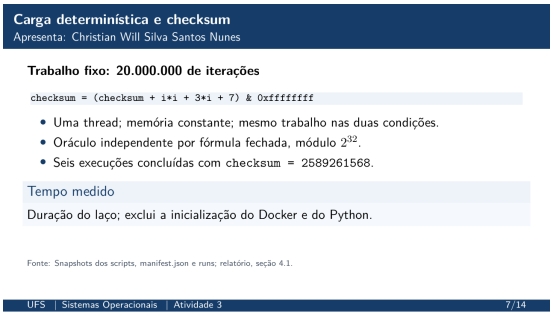
O Docker informou os PIDs 567 e 612 no host do daemon. Dentro de cada contêiner, o processo inicial tinha PID um. Não existe contradição: a numeração é relativa ao PID namespace.

As três linhas seguintes mostram handles distintos de PID, mount e network entre A e B. Isso sustenta a separação dessas visões na coleta. Os números são identificadores daquele momento, não valores fixos que outra reprodução precisa repetir.

Essa evidência descreve isolamento de visão. A capacidade de CPU e os demais limites serão tratados pelos cgroups. Christian vai mostrar como garantimos que as seis execuções fizeram o mesmo trabalho e como conferir os registros.

Slide 07 - Carga determinística e checksum

Janela 04:30-05:15 | Duração-alvo 00:45



EM TELA - não ler em voz alta

Christian assume. Apontar N fixo, expressão do checksum e oráculo independente. Não tratar a carga como inferência de LLM.

Próxima ação

Avançar ao slide 8; Christian continua.

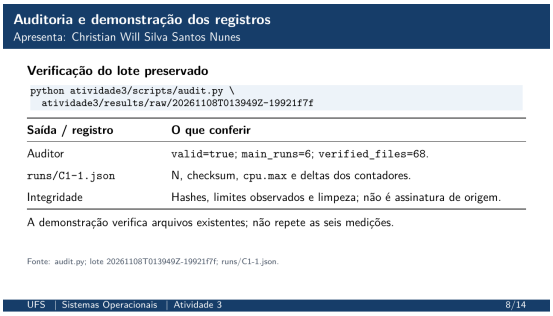
FALA - Christian Will Silva Santos Nunes (04:30-05:15)

Eu sou Christian. A carga executa vinte milhões de iterações de um cálculo aritmético, em uma thread e com memória constante. Cada iteração acumula i ao quadrado, mais três i, mais sete, com resultado limitado a trinta e dois bits.

O checksum esperado também é calculado por uma fórmula fechada independente do laço. Nas seis execuções, o valor foi dois bilhões, quinhentos e oitenta e nove milhões, duzentos e sessenta e um mil, quinhentos e sessenta e oito. A quantidade de trabalho e o resultado correto são comuns às duas condições; a quota é a variável comparada.

Slide 08 - Auditoria e demonstração dos registros

Janela 05:15-06:15 | Duração-alvo 01:00



EM TELA - não ler em voz alta

Executar o auditor do lote existente. Ampliar o terminal e mostrar valid=true, main_runs=6, verified_files=68. Depois abrir runs/C1-1.json. Se houver falha atual, explicar e abrir os registros preservados.

Próxima ação

Avançar ao slide 9 e passar a palavra a Marcos Vinícius Dantas Aguiar.

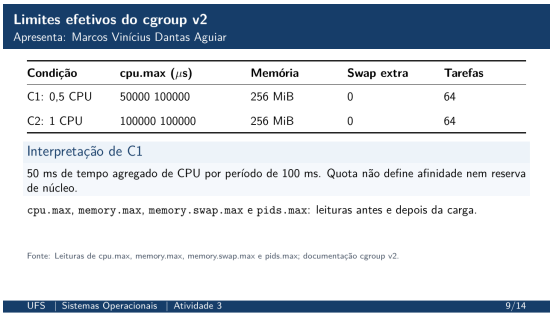
FALA - Christian Will Silva Santos Nunes (05:15-06:15)

Vou mostrar uma verificação dos arquivos já coletados. Este comando não repete o experimento: confere a integridade e a consistência do lote preservado. O resultado esperado para o lote publicado é valid igual a true, seis execuções principais e sessenta e oito arquivos verificados.

O auditor verifica hashes, checksums, limites e limpeza. No registro C1-1, podemos localizar o número de iterações, o checksum e cpu.max. Os contadores antes e depois permitem calcular o throttling. Uma auditoria de integridade não é uma assinatura independente da origem dos dados. Marcos vai explicar o significado dos limites locais e desses contadores.

Slide 09 - Limites efetivos do cgroup v2

Janela 06:15-07:00 | Duração-alvo 00:45



Limites efetivos do cgroup v2
Apresenta: Marcos Vinícius Dantas Aguiar

Condição	cpu.max (μs)	Memória	Swap extra	Tarefas
C1: 0,5 CPU	50000 100000	256 MiB	0	64
C2: 1 CPU	100000 100000	256 MiB	0	64

Interpretação de C1

50 ms de tempo agregado de CPU por período de 100 ms. Quota não define afinidade nem reserva de núcleo.

`cpu.max`, `memory.max`, `memory.swap.max` e `pids.max`: leituras antes e depois da carga.

Fonte: Leituras de cpu.max, memory.max, memory.swap.max e pids.max; documentação cgroup v2.

UFS | Sistemas Operacionais | Atividade 3 9/14

EM TELA - não ler em voz alta

Marcos assume. Apontar cpu.max nas duas condições e os três limites mantidos constantes.

Próxima ação

Avançar ao slide 10; Marcos continua.

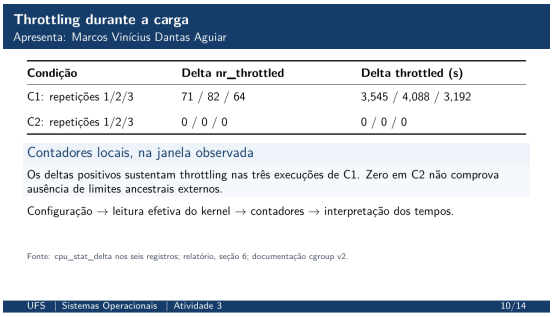
FALA - Marcos Vinícius Dantas Aguiar (06:15-07:00)

Eu sou Marcos. C1 tem quota de meia CPU e C2, de uma CPU. O arquivo cpu.max mostrou cinquenta mil por cem mil microssegundos em C1 e cem mil por cem mil em C2. Portanto, C1 dispõe de cinquenta milissegundos de tempo agregado de CPU por período de cem milissegundos.

Isso é quota, não afinidade ou reserva de um núcleo. Memória, swap adicional e tarefas ficaram fixos em 256 mebibytes, zero e sessenta e quatro. Essas condições foram confirmadas por leituras dos arquivos do kernel antes e depois do cálculo, além das opções usadas no Docker.

Slide 10 - Throttling durante a carga

Janela 07:00-07:45 | Duração-alvo 00:45



EM TELA - não ler em voz alta

Apontar os deltas das três repetições. Relacionar a quota com a restrição observada; passar para Sibele ao terminar.

Próxima ação

Avançar ao slide 11 e passar a palavra a Sibele Oliveira Cruz.

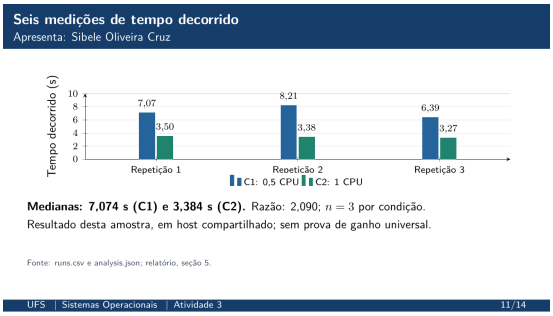
FALA - Marcos Vinícius Dantas Aguiar (07:00-07:45)

Para interpretar throttling, comparamos os contadores antes e depois da carga. Nas três execuções de C1, nr_throttled aumentou setenta e um, oitenta e dois e sessenta e quatro. O tempo associado à restrição também cresceu nas três.

Em C2, os deltas desses campos foram zero na janela observada. Isso não prova ausência de restrições em toda a hierarquia externa. Os contadores locais ajudam a explicar por que uma quota menor pode aumentar o tempo decorrido para o mesmo cálculo. Sibele vai mostrar os seis tempos e distinguir as métricas de memória.

Slide 11 - Seis medições de tempo decorrido

Janela 07:45-08:45 | Duração-alvo 01:00



EM TELA - não ler em voz alta

Sibele assume. Ler o gráfico por pares, apontar as medianas e n=3. Explicar a razão entre medianas.

Próxima ação

Avançar ao slide 12; Sibele continua.

FALA - Sibele Oliveira Cruz (07:45-08:45)

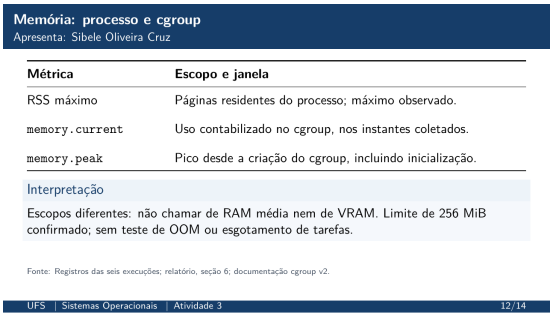
Eu sou Sibele. Cada par de barras representa uma repetição: azul para meia CPU e a segunda cor para uma CPU. As seis execuções concluíram o mesmo trabalho com o checksum correto. O tempo representado é o do cálculo, sem incluir a inicialização dos contêineres.

A mediana foi aproximadamente sete vírgula zero setenta e quatro segundos em C1 e três vírgula trezentos e oitenta e quatro em C2. A razão entre essas medianas foi dois vírgula zero noventa. As três observações de C1 foram maiores que as três de C2 nesta amostra.

São apenas três repetições por condição, em host compartilhado. Portanto, apresentamos os seis valores e a variação, sem concluir que qualquer carga terá exatamente esse ganho. Os deltas de throttling mostrados por Marcos ajudam a interpretar a diferença observada.

Slide 12 - Memória: processo e cgroup

Janela 08:45-09:25 | Duração-alvo 00:40



EM TELA - não ler em voz alta

Distinguir RSS máximo, memory.current e memory.peak. Ao terminar, passar para Silas, que segue até o encerramento.

Próxima ação

Avançar ao slide 13 e passar a palavra a Silas Santos da Silva.

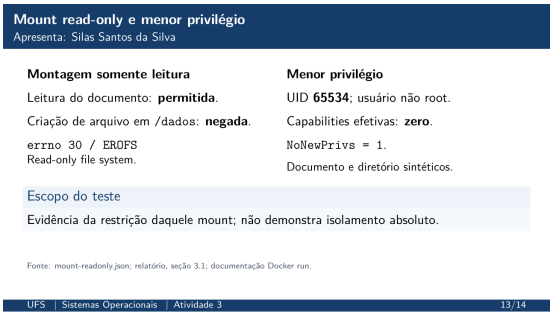
FALA - Sibele Oliveira Cruz (08:45-09:25)

As métricas de memória têm escopos diferentes. O RSS máximo descreve páginas residentes do processo. memory.current mostra o uso contabilizado do grupo nos instantes coletados. memory.peak cobre o pico desde a criação do cgroup, inclusive a inicialização.

Esses valores não devem ser chamados de RAM média, nem confundidos com memória de GPU. O limite de 256 mebibytes foi confirmado, mas não provocamos esgotamento de memória. Silas assume agora para mostrar a restrição de escrita e encerrar com os limites das conclusões.

Slide 13 - Mount read-only e menor privilégio

Janela 09:25-10:15 | Duração-alvo 00:50



EM TELA - não ler em voz alta

Silas assume. Mostrar leitura permitida e escrita negada. No vídeo, abrir mount-readonly.json e apontar errno, UID e controles.

Próxima ação

Avançar ao slide 14; Silas continua.

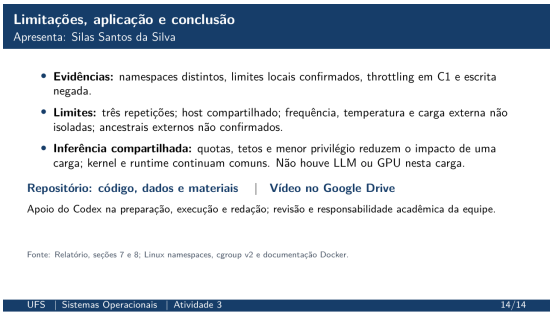
FALA - Silas Santos da Silva (09:25-10:15)

Eu sou Silas. Criamos um documento sintético e montamos seu diretório como somente leitura. A leitura funcionou. Ao tentar criar um novo arquivo no mesmo diretório, a operação foi negada com errno trinta, EROFS: sistema de arquivos somente leitura.

O processo também executou com UID 65534, capabilities efetivas zeradas e NoNewPrivs igual a um. A restrição da montagem soma-se aos controles de menor privilégio. O teste comprova essa negação de escrita no mount observado. Ele não demonstra segurança absoluta, porque kernel, runtime e outros componentes permanecem compartilhados.

Slide 14 - Limitações, aplicação e conclusão

Janela 10:15-11:00 | Duração-alvo 00:45



EM TELA - não ler em voz alta

Silas permanece até o encerramento. Mostrar limitações, conclusão e links. Não retornar a apresentadores anteriores.

Próxima ação

Encerrar a exposição e abrir o período de perguntas.

FALA - Silas Santos da Silva (10:15-11:00)

A amostra tem três repetições por condição em um host compartilhado. Frequência, temperatura e carga externa não foram isoladas, e não confirmamos os limites ancestrais externos pela visão disponível.

Em um servidor de inferência, quotas, tetos de memória e tarefas, corpus somente leitura e menor privilégio podem reduzir o impacto de uma carga. Este experimento, porém, não executou LLM ou GPU.

Concluímos que houve namespaces distintos, limites locais confirmados, throttling em C1 e escrita negada. O repositório reúne dados e materiais, e o vídeo está no link indicado. Declaramos o apoio do Codex e a responsabilidade de revisão da equipe. Obrigado; estamos disponíveis para perguntas.

Operação da demonstração

Apoio ao slide 8. Esta página orienta Christian e a pessoa que controla a tela.

Preparar antes de apresentar ou gravar

Abrir o repositório na sua raiz. Usar um ambiente em que python execute a versão configurada. No computador desta equipe, os ensaios anteriores usaram o ambiente Python no Ubuntu WSL.

Conferir o comando antes do ensaio e deixar o terminal com fonte grande. Não instalar programas, baixar imagem ou executar novamente a matriz durante a apresentação.

Deixar dois arquivos já abertos no editor: runs/C1-1.json e mount-readonly.json, nos caminhos completos abaixo. Esses arquivos registram observações anteriores; não são eventos que estão acontecendo agora.

Comando completo em uma única linha

```
python atividade3/scripts/audit.py atividade3/results/raw/20261108T013949Z-19921f7f
```

No Bash, a quebra de linha do slide usa uma barra invertida. A versão acima pode ser colada como um único comando. O auditor só verifica o lote existente; não precisa recriar os contêineres.

O que apontar na tela

1. Saída do auditor

Apontar valid=true, main_runs=6 e verified_files=68. Se a execução atual falhar, explicar o erro e abrir os registros preservados; não afirmar sucesso atual.

2. Registro C1-1

Apontar iterations=20000000, checksum=2589261568, correct=true, cpu.max="50000 100000" e cpu_stat_delta. Em C1-1, nr_throttled aumentou 71 e throttled_usec aumentou 3544785.

3. Mount read-only

Apontar content, write_denied=true, errno=30, uid=65534 e os controles de privilégio. Usar esse arquivo principalmente no vídeo; na apresentação, respeitar o minuto reservado à demonstração.

Arquivos para deixar abertos

`atividade3/results/raw/20261108T013949Z-19921f7f/runs/C1-1.json`

`atividade3/results/raw/20261108T013949Z-19921f7f/mount-readonly.json`

Plano de contingência: se o terminal não responder, mostrar os arquivos locais e explicar que são registros da coleta. Na apresentação, passar ao slide 9 e entregar a palavra a Marcos. No vídeo, corrigir o ambiente antes de refazer o trecho, identificando a auditoria como verificação de uma coleta anterior.

Apoio para as perguntas

Reservar 3-5 minutos após a exposição. As respostas abaixo são um apoio de estudo, não falas obrigatórias.

Iasmin - Onde está o kernel compartilhado?

No ambiente Linux intermediário da infraestrutura WSL2 em que executa o daemon. As imagens fornecem o espaço de usuário; cada contêiner não inicia seu próprio kernel.

Pedro - Como dois processos podem ter PID 1?

A numeração é relativa a cada PID namespace. Na visão informada pelo daemon, os processos tinham PIDs diferentes: 567 e 612.

Marcos - Meia CPU significa meio núcleo reservado?

É uma quota de tempo agregado por período. Não estabelece afinidade nem reserva de núcleo. Em C1, cpu.max mostrou 50000 100000 microssegundos.

Christian - Como sabemos que fizeram o mesmo trabalho?

N ficou congelado em vinte milhões de iterações. Todas concluíram o laço e concordaram com o checksum calculado por fórmula independente.

Sibele - Por que a razão não foi exatamente dois?

Há três observações por condição e interferência não isolada do host. A razão de medianas 2,090 é o resultado observado, sem provar um ganho universal.

Sibele - Por que RSS difere de memory.current?

São escopos e contabilizações distintos. RSS é do processo; memory.current é do grupo nos instantes coletados; memory.peak cobre a vida do cgroup.

Silas - EROFS comprova segurança absoluta?

Comprova a restrição daquele mount, junto com a opção ro observada. Kernel, runtime e host continuam compartilhados e exigem proteção e atualização.

Iasmin - Os ancestrais estavam sem limite?

Não foi possível confirmá-los pela visão de proc disponível. A ausência de acesso foi registrada como limitação, sem transformá-la em um valor ilimitado.

Bruno pode explicar a organização e a declaração de IA. A equipe deve distinguir apoio automatizado de contribuição efetivamente realizada e não inventar o identificador do modelo quando não estiver verificado.